Setup Guide

FPGA Matrix-Multiplication Accelerator on the ZCU102

For project groups continuing the TeraNova – FPGA Acceleration project

Written by	Kai Janssen (4994817)
Version	7 October 2026
Platform	AMD ZCU102 (Zynq UltraScale+ `xczu9eg-ffvb1156-2-e`)
Tools	Vivado / Vitis / Vitis HLS 2025.1.1
Workspace	`C:\PLD`

This guide is kept in `C:\PLD\SETUP_GUIDE.pdf`; its LaTeX source is `C:\PLD\report\setup_guide.tex`.

Contents

1 What this project is **2**

2 What you need **2**

2.1 About the licence 2

3 Folder layout **3**

4 First-time setup **3**

5 Common workflows **4**

5.1 Changing only the software 4

5.2 Changing `RAM_INTERFACE` (HLS) 4

5.3 Changing a VHDL IP (e.g. the partitioner) 5

5.4 Building the hardware and exporting the `.xsa` 5

5.5 Updating the Vitis platform after a hardware build 6

5.6 Running on the board 6

6 Simulation **7**

6.1 HLS 7

6.2 RTL simulation of the complete PL data path (GHDL) 7

6.3 Reference results (October 2026) 7

7 Known pitfalls **7**

8 Handing the project over **8**

9 More information **8**

1 What this project is

TeraNova measures nano-structures on wafers by shining a laser at them and solving an inverse-scattering problem. The question this project investigates is whether moving those calculations to an FPGA and accelerating them in hardware reduces the run time of the system.

The current hardware demonstrator multiplies two 50×50 matrices of 64-bit integers on the programmable logic (PL) of a Zynq UltraScale+ MPSoC, controlled by a bare-metal program on the ARM processor (PS):

1. the CPU writes matrices A and B into four block RAMs (`Part0`–`Part3`);
2. the CPU toggles a start GPIO;
3. `matrix_partitioner_B` reads the matrices in 5×5 blocks;
4. `MatMul5x5` multiplies two blocks; the accumulator sums 10 partial products per output block;
5. `RAM_INTERFACE` (HLS) writes each finished 5×5 result block to the PL-side DDR4 at `0x4_0000_0000`;
6. when the whole 50×50 result has been written, `done` raises an interrupt (and is also readable on GPIO channel 2); the CPU reads the result back, checks it and prints timing.

For the history of the design, the problems found and the decisions taken, read the documents in `C:\PLD\report` (Section 9).

2 What you need

Item	Details
Windows PC	At least 16 GB RAM (more speeds up builds). The Vivado/Vitis installation needs several tens of GB of disk space; the project itself is about 1.3 GB zipped.
Vivado + Vitis **2025.1.1**	Exactly this version. Install with *Zynq UltraScale+ MPSoC* device support and the *cable drivers*, to `C:\Xilinx\2025.1.1`. Vitis HLS is included in Vitis.
Licence for `xczu9eg`	Needed only for synthesis, implementation and bitstream generation in Vivado. The free edition does not cover this device. See Section 2.1.
ZCU102 board	With its 12 V supply and two micro-USB cables (JTAG and UART).
CP210x USB-UART driver	From Silicon Labs; needed to see the board's serial output.
Serial terminal	The Vitis Serial Monitor, or PuTTY.
GHDL + Python 3 (optional)	Only for the stand-alone RTL simulation in `C:\PLD\sim`.

2.1 About the licence

The group's Vivado licence for the `xczu9eg` is **node-locked**: it is tied to one PC and registered to one group member. Everything except the hardware build works without it: HLS (including C synthesis for this device), block-design editing and validation, all simulations, Vitis software builds, and programming and running the board.

Generate Output Products also needs the licence: with the default *Out of context per IP* setting it synthesises every IP that has no cached result, and each of those runs fails without a licence (see Section 5.4). Opening the project, *Report IP Status* and *Validate Design* do not need it.

A hardware build is only needed when the *hardware* changes. Changes to the C application never need one. Practical options when only one PC holds the licence: build on that PC (remote access or a copy of `C:\PLD`), or ask the supervisor about re-hosting the licence to a shared build PC.

3 Folder layout

Important. Everything must be at exactly `C:\PLD`. The Vivado project and the Vitis workspace contain absolute paths to this location; any other location breaks the projects.

Path (under `C:\PLD`)	Content
`S7_integration\`	Vivado project (`S7_integration.xpr`); block design `accu_to_DDR.bd`. "S7" is only a name: the target is the ZCU102.
`S7_integration_vit\`	Vitis workspace: platform `interrupt_rerun_plat` and application `interrupt_rerun_test`.
`S7_integration_vit\interrupt_rerun_test\src\helloworld.c`	The application: loads the matrices, starts the PL, waits for the interrupt, checks and times everything.
`ip_repo\`	Custom IP repository, one folder per IP: `RAM_INTERFACE`, `matrix_partitioner_B`, `MatMul5x5_Acc`, `MatMul5x5_PAR_IF`, `matrix_partitioner_old` (unused). This is the *only* IP repository the project uses.
`hls\RAM_INTERFACE\`	HLS source, testbench, configs and build scripts of the result writer (Section 5.2).
`scripts\`	Vivado Tcl scripts for packaging, block-design updates and the hardware build.
`sim\`	GHDL testbench of the complete PL data path.
`report\`	Analysis report, project log and the source of this guide.
`backup\`	Older IP versions, the previous `.xsa`, and dated full copies of the projects.
`README_UPGRADE.md`	The original upgrade plan of October 2026 (historical; this guide supersedes it).

4 First-time setup

1. **Get the files.** Unpack the hand-over archive so that the folder is `C:\PLD` (check that `C:\PLD\S7_integration\S7_integration.xpr` exists, not `C:\PLD\PLD\...`). Use 7-Zip; Windows' built-in extractor is slow and can fail on long paths.

2. **Install the tools.** Install Vivado/Vitis 2025.1.1 (see the table above). Then run, as administrator:

   ```
   C:\Xilinx\2025.1.1\data\xicom\cable_drivers\nt64\install_drivers_wrapper.bat
   ```

   and install the Silicon Labs CP210x driver.

3. **Open the Vivado project.** Start Vivado 2025.1.1 → *Open Project* → `C:\PLD\S7_integration\S7_integration.xpr`. Then *Reports* → *Report IP Status*. Expected: all IPs up to date, none locked.

   - Many `[Board 49-26] cannot add Board Part ...` warnings are normal: they concern evaluation boards of device families that are not installed.
   - "generated file not found ... Please regenerate" warnings (e.g. for `accu_to_DDR_axi_smc_0`) only mean that the cached synthesis results of that IP are missing. They are recreated by the hardware build on the licensed PC; on an unlicensed PC you cannot clear them.
   - "Up-to-date (discontinued)" for three Constant IPs inside the SmartConnects is normal and needs no action.
   - The IP Catalog must list only `C:\PLD\ip_repo` as user repository (*Settings* → *IP* → *Repository*). Older copies of the project also listed `../Cezar_repo`, a folder that was never part of the hand-over; it was removed from the project on 7 October 2026. If you see it, remove it with `set_property ip_repo_paths C:/PLD/ip_repo [current_project]` followed by `update_ip_catalog`.
   - If *all* IPs are locked with "Newer software version", you are not on 2025.1.1.
   - If only `RAM_INTERFACE_0` is locked, the IP was rebuilt after the block design was saved: run `2b_upgrade_after_rebuild.tcl` (Section 5.2).

4. **Open the Vitis workspace.** Start Vitis 2025.1.1 → *Open Workspace* → `C:\PLD\S7_integration_vit`. Select the component `interrupt_rerun_test` in the *Flow* panel and click *Build*. It must finish with a size summary and no errors.

5. **Check the board connection** (Section 5.6).

5 Common workflows

Pick the workflow that matches what you change. Always make a dated copy of `C:\PLD\S7_integration` and `C:\PLD\S7_integration_vit` in `C:\PLD\backup` before larger changes.

5.1 Changing only the software

1. Edit the files in `interrupt_rerun_test\src`. Compiler options (currently `-O2`) are in `UserConfig.cmake`.
2. Build `interrupt_rerun_test` in Vitis and run it on the board (Section 5.6). No licence or hardware build needed.

Important. If you edit a source file outside Vitis, close it in Vitis first. Otherwise Vitis may save its older copy over your change.

5.2 Changing `RAM_INTERFACE` (HLS)

The source is `RAM_INTERFACE.cpp/.hpp` in `C:\PLD\hls\RAM_INTERFACE`; the testbench is `RAM_INTERFACE_TEST.cpp`. The block is free-running (`ap_ctrl_none`): it reads one 5×5 block from an `ap_fifo` stream (connected to the accumulator's `C_valid`/`C_ready`), writes it through `m_axi` to DDR4, and raises `done` after the last block.

1. **Build the IP.** In a Command Prompt:

```
cd /d C:\PLD\hls\RAM_INTERFACE
build_ip.bat > build_log.txt 2>&1
```

This runs C simulation, C synthesis and packaging, and installs the IP in `C:\PLD\ip_repo\RAM_INTERFACE` (the previous one goes to `C:\PLD\backup\RAM_INTERFACE_old`). `build_log.txt` must end with `DONE`. The synthesis report is `work\hls\syn\report\RAM_INTERFACE_csynth.rpt`; check that the estimated clock period is below 10 ns.

2. **Co-simulate the RTL.**

```
run_cosim.bat > cosim_log.txt 2>&1
```

The log must contain `C/RTL co-simulation finished: PASS`. Co-simulation does not support this `ap_ctrl_none` design directly, so `run_cosim.bat` builds a variant with an `ap_ctrl_hs` handshake (macro `COSIM_HS`, set in `hls_config_cosim.cfg`, work folder `work_cosim`). The data path and memory writes are identical; the packaged IP is not affected. Keep the `#ifdef COSIM_HS` block in `RAM_INTERFACE.cpp`.

3. **Update the block design.** In the Vivado Tcl console, with the project open:

```
source C:/PLD/scripts/2b_upgrade_after_rebuild.tcl
```

This upgrades the IP in place, keeps all connections, validates and saves. Validation must give no errors.

4. Continue with the hardware build (Section 5.4).

Important. `2b_upgrade_after_rebuild.tcl` only works if the ports did not change. If you add, remove or rename ports, you must reconnect the cell yourself; `2_update_block_design.tcl` shows how (it deletes the cell, recreates it and wires every pin). Do not run script 2 on a block design that is already wired correctly.

5.3 Changing a VHDL IP (e.g. the partitioner)

1. Edit the VHDL in its `C:\PLD\ip_repo` folder (keep a copy of the old file in `C:\PLD\backup`). Simulate it first with the GHDL testbench (Section 6).
2. Raise the core revision so that Vivado offers an *upgrade*: copy `scripts\1_repackage_partitioner.tcl`, change the `set_property core_revision 4` line to the next number (and the `component.xml` path for another IP), and source it in Vivado.
3. Upgrade the IP in the block design (*Report IP Status* → *Upgrade Selected*, or adapt `2b_upgrade_after_rebuild.tcl`), validate and save.

Important. Never accept an "Upgrade IP" that goes to a *lower* revision than the one in the design. That happened with the original hand-over: the archived IPs were older than the ones used in the project.

5.4 Building the hardware and exporting the `.xsa`

On the PC with the licence, with the project open in Vivado:

```
source C:/PLD/scripts/3_build_and_export.tcl
```

This backs up the old `.xsa` to `C:\PLD\backup`, generates the output products, runs synthesis and implementation up to the bitstream (this takes a long time), prints the worst negative slack and writes `C:\PLD\S7_integration\interrupt_rerun.xsa`.

- The worst negative slack (WNS) must be ≥ 0. If not, timing is not met and the hardware cannot be trusted.
- `ERROR: [Common 17-345] A valid license was not found ... 'xczu9eg'` means this PC has no licence. The same error appears when you run *Generate Output Products* on an unlicensed PC: the out-of-context runs of the IPs without a cached result fail (on 7 October 2026: `axi_smc_0`, `matrix_partitioner_B_0` and `RAM_INTERFACE_0`) and Vivado shows "Multiple block runs Failed". Nothing in the project is damaged; build on the licensed PC.
- The message "Failed to create directory 'C' " at start-up is harmless.
- Twelve `[BD 41-2383] Width mismatch` critical warnings (six connections, each reported twice: by validation and by generation) come from the original design and are harmless. They concern the address path partitioner → `mult_gen` → BRAM: `bram_addr_A/B` (12 bits) to `mult_gen_*/A` (18 bits), and `mult_gen_*/P` (21 bits) to `Part0..3/addrb` (32 bits). The narrower side always covers the full address range (12 bits = 4096 values for 2500 elements), and the unconnected upper bits are tied to 0 in synthesis, which is the correct value.

5.5 Updating the Vitis platform after a hardware build

1. In Vitis, open `interrupt_rerun_plat` → *Settings* (`vitis-comp.json`) → update the hardware specification to `C:\PLD\S7_integration\interrupt_rerun.xsa`, then build the platform.
2. Build `interrupt_rerun_test`.
3. Check that `interrupt_rerun_test\_ide\bitstream\interrupt_rerun.bit` has today's date. If not, copy `interrupt_rerun_plat\export\interrupt_rerun_plat\hw\interrupt_rerun.bit` over it. The run configuration programs the FPGA from this file.

5.6 Running on the board

1. Set boot-mode switch **SW6 to JTAG** (all four switches ON). Check this against the label on the board.
2. Connect the JTAG and UART micro-USB cables and the 12 V supply; switch on with **SW1**.
3. Device Manager should show four Silicon Labs CP210x COM ports. The *lowest* number is PS UART0, where the program prints.
4. Open a terminal on that port: 115 200 baud, 8 data bits, no parity, 1 stop bit, no flow control.
5. Close Vivado's Hardware Manager (it occupies the JTAG cable), then in Vitis select `interrupt_rerun_test` and click *Run*.

Expected output: for each of two runs, the BRAM load time, PL compute time, read-back time and a correctness check, plus the CPU-only reference time, ending with `ALL RUNS PASSED`. If the interrupt does not arrive, the program times out after 1 s, reads `done` on GPIO channel 2 and reports whether the PL finished. This tells you whether the problem is in the PL or in the interrupt path.

6 Simulation

6.1 HLS

C simulation is part of `build_ip.bat`; C/RTL co-simulation is `run_cosim.bat` (Section 5.2). Both need no licence and no board.

6.2 RTL simulation of the complete PL data path (GHDL)

`C:\PLD\sim` contains `tb_chain.vhd`, which connects a BRAM model, the partitioner, the multiplier, the accumulator and a model of the `RAM_INTERFACE` handshake. `run.py` writes the input matrices, runs GHDL and compares the 50×50 result with Python. In a Command Prompt with `ghdl` and `python` on the PATH:

```
cd /d C:\PLD\sim
for %f in (MatMul5x5.vhd MatMul5x5_IF_IN.vhd MatMul5x5_Accumulator.vhd matrix_to_ports
    .vhd matrix_partitioner_B.vhd tb_chain.vhd) do ghdl -a --std=08 -frelaxed %f
ghdl -e --std=08 -frelaxed tb_chain
python run.py
python run.py -gWRITE_BUSY_CYCLES=1500
```

The last line simulates a slow writer (back-pressure). All cases must report PASS. The VHDL files in `C:\PLD\sim` are copies; if you change an IP, copy the new VHDL here as well. The testbench can also be run in the Vivado simulator (add the files, set `tb_chain` as top, put `a_b.hex` in the simulation run directory).

6.3 Reference results (October 2026)

Test	Result
HLS C simulation, 2 complete matrices	PASS
HLS C/RTL co-simulation, 2×100 blocks	PASS, ≈78 cycles per block
`RAM_INTERFACE` synthesis at 100 MHz	7.3 ns estimated (Fmax ≈137 MHz)
GHDL, test data / random 64-bit / slow writer	PASS / PASS / PASS
PL run time for one 50×50 product (GHDL)	101 008 cycles ≈ 1.01 ms at 100 MHz
Board run	not yet done at the time of writing

7 Known pitfalls

Pitfall	What to do
Wrong tool version	Only 2025.1.1. Other versions lock every IP and offer upgrades you do not want.
Project not at `C:\PLD`	Absolute paths break. Move it back rather than editing paths.
Upgrade offered to an older IP revision	Do not accept it; find out why the repository holds an older version.
`RAM_INTERFACE_0` locked after `build_ip.bat`	Expected: a new HLS build has a new revision. Run `2b_upgrade_after_rebuild.tcl`.
Co-simulation refuses to start (`COSIM 212-345`)	You ran co-simulation with `hls_config.cfg`. Use `run_cosim.bat`.

Pitfall	What to do
Edit lost in `helloworld.c`	Vitis had the file open. Close files in the IDE before editing them elsewhere.
Old bitstream on the board	Check the date of `_ide\bitstream\interrupt_rerun.bit` after updating the platform.
No serial output	Wrong COM port (use the lowest of the four), wrong baud rate, or the CP210x driver is missing.
JTAG busy / board not found in Vitis	Close Vivado's Hardware Manager; check SW6 and the JTAG cable.
Licence error during synthesis, or "Multiple block runs Failed" after *Generate Output Products*	This PC has no licence for the `xczu9eg`. Build on the licensed PC (Section 2.1).
Extra IP repository in the project (e.g. `Cezar_repo`)	Remove it (Section 4, step 3). Only `C:\PLD\ip_repo` is needed.

8 Handing the project over

1. Close Vivado and Vitis.
2. Zip the complete `C:\PLD` folder with 7-Zip. To save space you may leave out folders that are regenerated automatically: `S7_integration\S7_integration.runs`, `S7_integration\S7_integration.cache`, `hls\RAM_INTERFACE\work`, `hls\RAM_INTERFACE\work_cosim` and the Vitis `build` folders. Keep everything else, including `S7_integration.gen` and `backup`.
3. Update this guide and the project log with what you changed, and note the board results.
4. Tell the next group who holds the licence and on which PC.

9 More information

- `C:\PLD\report\previous_group_analysis.tex` – analysis of the original design: data flow, cycle and resource budgets, bottlenecks and proposed improvements (burst transfers, removing wait states, data reuse).
- `C:\PLD\report\project_log_kai_janssen.tex` – chronological log of the take-over, with all decisions and test results.
- `C:\PLD\README_UPGRADE.md` – the October 2026 plan for restoring the two IPs.
- `C:\PLD\sim\README.md` – details of the GHDL testbench.